Distributed Causal Observability for Multi-Agent Workloads

Tegan Jain

November 2026

Abstract Your abstract here.

Contents

0. Front matter 1
1. Introduction 2
1.1 Motivation 2
2. State of Agentic AI Research [you] 2
3. Background: observability in distributed systems 2
3.1 Tracing basics 2
4. Related work: agent observability 3
4.1 AgentSight 3
5. Failure model and causal framework 3
5.1 Failure model 3
6. Testbed and ground truth 3
6.1 Workload and topology 3
7. Observability framework design 4
7.1 Overview and layers 4
8. Evaluation 4
8.1 Setup, baselines, metrics 4
9. Engineering findings on AgentSight and local-model support 4
9.1 Build and portability issues 4
10. Discussion and threats to validity 5
11. Future work 5
References 5
Appendix 5
RQ traceability (fill in before the defence) 5

0. Front matter

- Title, abstract (write last: gap, approach, what was measured), keywords

1. Introduction

1.1 Motivation

- Agents moving into infrastructure operations; many LLM calls per task, so cost and energy matter, which argues for local or small models [you]
- Operators need to know which agent action caused an SLO violation, and whether a claimed action really happened ### 1.2 Problem statement
- State the objective verbatim: cross-node causal links between agents, infrastructure effects and SLO outcomes; adaptive eBPF telemetry; outcome verification ### 1.3 Gap
- AgentSight: single agent, heuristic correlation, no ground truth, observer cost unquantified
- AgentLoupe: single host, propagation fidelity unmeasured by its own admission
- llmmas-otel: app-level instrumentation, no eBPF; multi-host not addressed ⚠
- Nobody specifies or measures what the observer LLM is shown (RQ3) ### 1.4 Research questions
- **RQ1** [you]: not in the text I saw. My guess: what failures can occur and which of them the framework promises to detect, localize and explain
- **RQ2**: can delayed and indirect effects of agent decisions be traced at manageable overhead?
- **RQ3**: what should the observer agent be shown, and from where does that model come?
- **RQ4**: how can events be replayed and counterfactuals answered from observability data? ⚠ Scope this carefully (see §5.3 and §7.6) ### 1.5 Contributions
- 3 or 4 bullets, each pointing to a section and an RQ ### 1.6 Scope and non-goals
- Agentic workloads only. "Nodes" are emulated on one machine
- Classical workloads in the same cluster: noted in §11, not built

2. State of Agentic AI Research [you]

Placed here because it motivates the workload model before the observability background. ### 2.1 Agent architectures and frameworks - Single-agent loops (tool use), multi-agent patterns (planner/worker/critic, debate, pipelines), common frameworks ### 2.2 Communication and tool protocols - MCP (tools), A2A (agent to agent). Transport, streaming, what is observable on the wire ⚠ ### 2.3 Serving and runtime - LLM serving for agent workloads (Parrot, Autellix), shared inference servers as a contention point ### 2.4 Failures and reliability of agent systems - MAST taxonomy, Who&When (53.5% agent-level, 14.2% step-level attribution) ⚠ ### 2.5 Local and small models - Tool-calling reliability at 7B, context-window limits, cost and energy argument [you] - Your own data: opencode failures and the model comparison ### 2.6 Agents in infrastructure operations - Deployment of agents in network and cloud operations; why observability matters there [you]

3. Background: observability in distributed systems

3.1 Tracing basics

- Dapper: spans, trace IDs, sampling. Context propagation as the central requirement ### 3.2 Causality and time
- Happened-before (Lamport), clocks, clock skew, why per-host timestamps can't be compared directly ### 3.3 Zero-instrumentation tracing
- eBPF, DeepFlow-style inference of links from network signals ### 3.4 Sampling and collection strategies
- Head, tail and retroactive sampling (Hindsight: local buffers, triggers, breadcrumbs). ⚠ Your plan lists Hindsight twice (once as "logging service"); it is the same paper ### 3.5 Root-cause analysis

- Causal-inference RCA benchmark study: why not statistical causal discovery
- Reading status and gaps: [you]

4. Related work: agent observability

4.1 AgentSight

- Boundary tracing; three signals (lineage, temporal proximity, argument matching); 100–500 ms window unjustified; no ground truth; multi-agent case study is one qualitative session; overhead: 3 runs, mean ⚠ ### 4.2 AgentLoupe
- Kernel-resident chain IDs propagated through fork, inode, socket handoffs; single host; two case studies; fidelity unmeasured ⚠ re-read how it decides a prompt is new ### 4.3 llmmas-otel
- Fault taxonomy at LLM, tool and A2A boundaries; context in message carrier; ChatDev evaluation ### 4.4 Failure attribution from logs
- Who&When, MAST ### 4.5 Comparison table and positioning
- Rows: systems. Columns: zero-instrumentation, multi-agent, cross-host, ground truth, overhead measured (time/tokens), adaptive, local-model support
- Note on earlier work being lower impact (predates AgentSight)

5. Failure model and causal framework

5.1 Failure model

- Triple: root cause, propagation channel, symptom
- Root causes: model failures (hallucination, loops, truncation), resource, tool/state, message, coordination
- Channels: message, contention, shared state, delayed/persistent, semantic ### 5.2 What the framework claims
- Provenance (happens-before) and attribution (blame along edges)
- Edge labels: exact, inferred, semantic ### 5.3 Counterfactuals (RQ4)
- Counterfactual runs in the testbed are the grading oracle
- ⚠ Decide whether RQ4 claims (a) rerun-based oracle, or (b) a what-if estimate computed on the captured graph. They are different claims ### 5.4 Promise matrix
- Faults × channels; cells: detect / localize / explain / out of scope; justify each out-of-scope cell ### 5.5 Timing and staleness
- The observed system keeps moving while analysis runs; so snapshot at trigger and keep the LLM off the real-time path

6. Testbed and ground truth

6.1 Workload and topology

- Planner, worker, critic; shared Ollama; shared files; fan-out and retry loop ### 6.2 Communication
- Per-agent HTTPS servers, A2A-style messages; why (TLS-plaintext visibility); what isn't covered (streaming, HTTP/2, gRPC, brokers) ### 6.3 Model and context-window choices
- qwen2.5, 8K context; why off-the-shelf agent harnesses failed on a 7B local model (system prompt size vs context) [measure: tokens of the opencode system prompt] ### 6.4 Fault injection
- Fault list with parameters; compare with llmmas-otel's boundary faults; add resource and shared-state faults they lack ⚠ ### 6.5 Ground-truth log

- Schema, no IDs on the wire, hash-joined message edges, fault-tagged events ### 6.6 SLO definition and reproducibility
- Calibration from baselines, seeds, paired fault-on/off runs, noise floor

7. Observability framework design

7.1 Overview and layers

- Capture, identity and clocks, edge builders, causal DAG, attribution, adaptive collection, observer LLM ### 7.2 Capture
- sslsniff and process probes; what each emits; truncation and loss behaviour ### 7.3 Identity and linking
- Linking ladder: infer by default, propagated trace ID if present, kernel labels within a host
- Cross-node exact hops by plaintext hash at both ends; semantic hops by content containment [measure] ### 7.4 The causal DAG
- Nodes: LLM calls, messages, file/process events. Edges: typed and labelled. DAG not tree
- Causal and machine IDs on every event ### 7.5 Storage and retroactive collection
- Record cheaply always, collect fully on trigger (SLO burn, error), follow breadcrumbs; query by ID on failure ### 7.6 Replay (RQ4)
- What can be replayed (seeded testbed), what can't (LLM nondeterminism, external effects) ### 7.7 Adaptive telemetry
- Tiers and triggers; overhead vs accuracy knob ### 7.8 The observer LLM (RQ3)
- Where the model comes from (local, small), what it sees, context budget
- Selecting the relevant subgraph instead of dumping events. Prompt design
- Your observation: one `requests` call touches dozens of files, all attributed to the LLM call; quantify events per call [measure] ### 7.9 Emulating a distributed deployment
- Namespaces or containers on one machine; injected clock offsets; what changes on real clusters

8. Evaluation

8.1 Setup, baselines, metrics

- Baselines: random, AgentSight correlator, request-level critical path
- Metrics: edge precision and recall per type, top-1/top-3 root cause, overhead, noise floor ### 8.2 AgentSight on your testbed
- Bridge, count mismatches, correlator precision on real traces. Don't cite synthetic-data numbers as results ### 8.3 Edge recovery (RQ1/RQ2) [measure] ### 8.4 Attribution against fault-on/fault-off ground truth [measure] ### 8.5 Overhead (RQ2)
- Time and CPU, plus tokens and context size (neither AgentSight nor AgentLoupe reports tokens ⚠), several runs with variance, not just means ### 8.6 Observer input size and quality (RQ3) [measure]
- Observer input tokens vs events per call; effect of subgraph selection on explanation quality ### 8.7 Adaptive collection (RQ2)
- Always-on vs always-cheap vs adaptive curve

9. Engineering findings on AgentSight and local-model support

9.1 Build and portability issues

- Clang inlining pushed a probe past the 512-byte BPF stack limit (fixed with `__noinline`); missing symbol; the bash `readline` probe fails where readline is a shared library ### 9.2 Local-model gap

- Source read: provider list covers hosted providers; the LLM-path list has no Ollama native `/api/chat`; token fields don't include Ollama's `prompt_eval_count`/`eval_count`; Ollama streams NDJSON, not SSE ⚠ check streaming
- Check whether Ollama's OpenAI-compatible `/v1` endpoint already works before claiming "no support" ### 9.3 Proposed contribution
- Upstream PR for local-model support; scope and test plan

10. Discussion and threats to validity

- Single machine; small model; synthetic tasks; few fault types; small samples
- Inferred edges and the FIFO assumption; observer faults (capture loss)
- Out-of-scope cells from the matrix

11. Future work

- Real multi-host deployment
- Mixed agentic and classical workloads: hundreds of containers with very different characteristics, shared contention between them, one platform for both (supervisor's point; mention only)
- HTTP/2, gRPC, streaming, brokers; learned adaptive policies; acting on attributions; larger models; semantic edges

References

- Verify each citation against the paper itself

Appendix

- Filled promise matrix, ground-truth schema, fault parameters, opencode failure transcripts, AGENTS.md

---

RQ traceability (fill in before the defence)

RQ	Sections	Evidence
RQ1	5, 8.3	
RQ2	7.3–7.5, 7.7, 8.3, 8.5, 8.7	
RQ3	7.8, 8.6	
RQ4	5.3, 7.6, 8.4	